

A Comprehensive Technical Review

# Modern Data Engineering & Architecture

Strategic Design for Enterprise Scalability and Insight

---

# | The Role of Data Engineering

## Designing the Backbone

Data engineering is the practice of designing and building systems for collecting, storing, and analyzing data at scale. It transforms raw data into a reliable foundation for business intelligence.

- ✓ **Data Ingestion:** Harvesting from diverse sources.
- ✓ **Infrastructure:** Scalable cloud & on-prem systems.
- ✓ **Transformation:** Cleansing and structuring logic.



# Storage: Lake vs. Warehouse

## Strategic Positioning

**Data Lake:** A central repository that allows you to store all your structured and unstructured data at any scale (Raw, Schema-on-Read).

**Data Warehouse:** A system used for reporting and data analysis. It stores structured data that has been cleaned and integrated (Refined, Schema-on-Write).

### Key Takeaway:

Lakes provide flexibility for discovery; Warehouses provide performance for production analytics.

# | Specialized Views: Data Marts



## Marketing Mart

Focused on customer acquisition, churn rates, and campaign ROI. Highly specific schema for rapid BI.



## Finance Mart

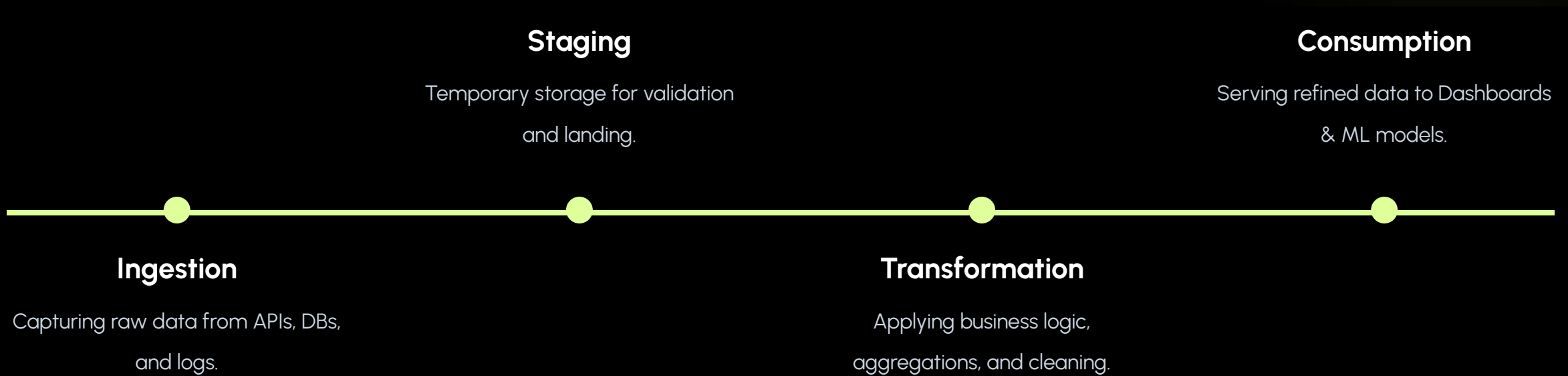
Tailored for profit & loss statements, taxation, and budget forecasting. High precision and auditability.



## Inventory Mart

Streamlined for supply chain logistics, stock levels, and vendor management performance.

# | The Data Pipeline Lifecycle



Pipelines ensure **Consistency**, **Reliability**, and **Automation** across the stack.

# | ETL vs. ELT Paradigms

| Feature        | ETL (Traditional)                | ELT (Modern Cloud)                              |
|----------------|----------------------------------|-------------------------------------------------|
| Process        | Transform before loading         | Load then transform in-target                   |
| Transformation | Done in a separate ETL engine    | Done using target DB power                      |
| Flexibility    | Rigid; defines schema early      | High; stores raw data for future needs          |
| Scalability    | Dependent on ETL server capacity | Natively scales with Cloud (Snowflake/BigQuery) |

# | SQL: The Universal Interface

100%

Platform Compatibility

SQL remains the standard across BigQuery, Snowflake, Redshift, and SparkSQL.

4th

Generation Language

Declarative syntax allows engineers to focus on 'what' to get, not 'how' to get it.

---

# | Advanced SQL Concepts



## Window Functions

Calculating aggregates over subsets without collapsing rows. Essential for **RANK()**, **LAG()**, and **MOVING\_AVG()**.



## CTEs & Recursion

Common Table Expressions (WITH clause) improve readability and allow for **Recursive** queries to navigate hierarchical data.

```
SELECT name, salary, RANK() OVER (PARTITION BY dept ORDER BY salary DESC) as rank FROM employees;
```



# | Database Normalization

Normalization is the process of structuring a relational database to reduce data redundancy and improve data integrity by ensuring dependencies are logical.

## 1

### 1st Normal Form

Ensures **atomicity**. Each column contains only one value, and there are no repeating groups of columns.

## 2

### 2nd Normal Form

Meets 1NF requirements and ensures all non-key attributes are **fully dependent** on the primary key.

## 3

### 3rd Normal Form

Meets 2NF requirements and removes **transitive dependencies**. Non-key columns depend only on the primary key.

---

\*Achieving 3NF is standard for OLTP systems to minimize update anomalies and storage waste.

# | Slowly Changing Dimensions (SCD)

SCD refers to techniques for managing how dimensional data (like customer addresses or product prices) changes over time in a data warehouse.

| Type   | Action         | History Retention | Best Use Case                  |
|--------|----------------|-------------------|--------------------------------|
| Type 0 | Original       | None (Fixed)      | Birthdays, Original Order IDs  |
| Type 1 | Overwrite      | No History        | Correcting Spelling Errors     |
| Type 2 | Add New Row    | Full History      | Tracking Career Titles/Regions |
| Type 3 | Add New Column | Partial History   | Previous vs. Current Salary    |

**i** **Type 2** is the industry standard for analytical systems, typically utilizing `Effective_Date`, `End_Date`, and `Is_Current` flags.

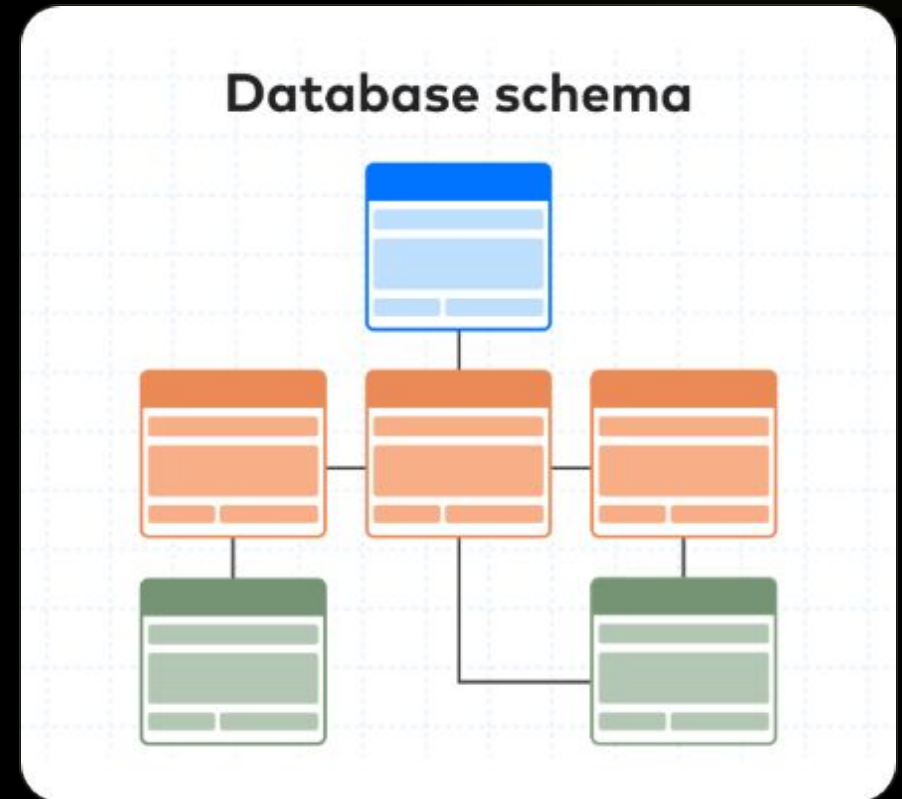
# Dimensional Modeling: Facts & Dimensions

## The Star Schema Philosophy

**Facts:** Quantitative data (measures) that are typically numeric and additive. (e.g., Sales Amount, Quantity, Temperature).

**Dimensions:** Descriptive data that provides context to the facts. (e.g., Date, Product Name, Store Location, Customer Segments).

- High Query Performance
- Easily understood by BI Tools
- Simple Joins



# | Structural Layers of Data Modeling



## Conceptual

High-level business entities and their relationships. Independent of any database technology.



## Logical

Adds attributes and defines primary/foreign keys.  
Technology-agnostic but highly detailed.



## Physical

The actual implementation:  
Tables, Columns, Data Types, Indexes, and Partitioning schemes.

# Comparative Analysis of Schema Types

| Metric        | Star Schema           | Snowflake Schema      | Data Vault 2.0               |
|---------------|-----------------------|-----------------------|------------------------------|
| Normalization | Denormalized          | Fully Normalized      | Highly Normalized (Hubs/Sat) |
| Query Speed   | Fastest (Few joins)   | Moderate (Many joins) | Complex for direct queries   |
| Redundancy    | High                  | Low                   | None                         |
| Adaptability  | Low (Rigid structure) | Moderate              | High (Modular history)       |

# Questions & Strategic Discussion

Thank you for your attention



strategy@enterprise-data.com [www.datainsights.com](http://www.datainsights.com)